Announce Finder

Name: Simu Darius

Group: 30435

Contents

1 Deliverable 1 **3**
- 1.1 Project Specification . . . 3
- 1.2 Functional Requirements . . . 3
- 1.3 Use Case Model . . . 3
- 1.4 UML Use Case Diagrams . . . 5
- 1.5 Supplementary Specification . . . 5
  - 1.5.1 Non-functional Requirements . . . 5
  - 1.5.2 Design Constraints . . . 6
- 1.6 Glossary . . . 6

2 Deliverable 2 **7**
- 2.1 Domain Model . . . 7
- 2.2 Conceptual Architecture . . . 8
- 2.3 Package Design . . . 9
- 2.4 Components and Deployment . . . 9

3 Deliverable 3 **10**
- 3.1 Design Model . . . 10
  - 3.1.1 Dynamic Behavior . . . 10
  - 3.1.2 Class Diagram . . . 10
  - 3.1.3 Data Model . . . 10
  - 3.1.4 System Testing . . . 10
  - 3.1.5 Future Improvements . . . 10
- 3.2 Conclusion . . . 10
- 3.3 Bibliography . . . 10

1 Deliverable 1

1.1 Project Specification

AnnounceFinder is a web-based application that aggregates product listings and announcements from multiple online marketplaces into a single unified search interface. The system crawls and scrapes data from both second-hand platforms as well as brand new retail websites, allowing users to search for any type of product or item. Results are presented in a consistent format including title, price, platform, image, and a direct link to the original listing. The user may choose how products are ordered: depending on price, platfrom, or type: new or used. Registered users can authenticate and save listings to a personal favorites list stored in a database.

1.2 Functional Requirements

- FR1: The system shall allow users to search for any product or item by entering a keyword in a search field.
- FR2: The system shall query multiple data sources simultaneously and aggregate the results.
- FR3: The system shall display search results including title, price, platform, image, and a link to the original listing.
- FR4: The system shall filter results to ensure relevance to the search query using keyword and fuzzy matching.
- FR5: The system shall allow users to choose how the items are ordered: based on price, platform or type: new or used.
- FR6: The system shall allow users to register and log in with a username and password.
- FR7: Authenticated users shall be able to save listings to a personal favorites list.
- FR8: Authenticated users shall be able to view and manage their saved favorites.
- FR9: The system shall respect robots.txt rules for all scraped websites.
- FR10: The system shall enforce rate limiting between requests to avoid overloading external servers.

1.3 Use Case Model

Use-Case: Search for listings
Level: User goal
Primary Actor: Guest or registered user
Main success scenario:

1. User enters a search query in the search field.
2. System sends the query to all registered scrapers.
3. Each scraper fetches results from its respective platform.
4. System filters results using keyword and fuzzy matching.
5. System aggregates and returns the results.
6. Results are displayed to the user with title, price, platform, and image.

Extensions:

1. A scraper fails or is blocked by robots.txt:
   (a) The scraper returns an empty list and logs the error.
   (b) Results from remaining scrapers are still displayed.
2. Query is empty:
   (a) System returns no results and prompts the user to enter a keyword.

Use-Case: Register and log in
Level: User goal
Primary Actor: Guest user
Main success scenario:

1. User navigates to the registration page.
2. User enters a username and password.
3. System validates the input and creates an account.
4. User logs in with their credentials.
5. System authenticates the user and starts a session.

Extensions:

1. Username already exists:
   (a) System notifies the user and prompts for a different username.
2. Password does not meet requirements:
   (a) System displays password requirements and asks the user to retry.

Use-Case: Save a listing to favorites
Level: User goal
Primary Actor: Registered user
Main success scenario:

1. User performs a search and receives results.
2. User clicks the "Save to Favorites" button on a listing.
3. System stores the listing details in the database linked to the user.
4. System confirms the listing was saved.

Extensions:

1. User is not logged in:
   (a) System redirects the user to the login page.
2. Listing already saved:
   (a) System notifies the user the listing is already in their favorites.

1.4 UML Use Case Diagrams

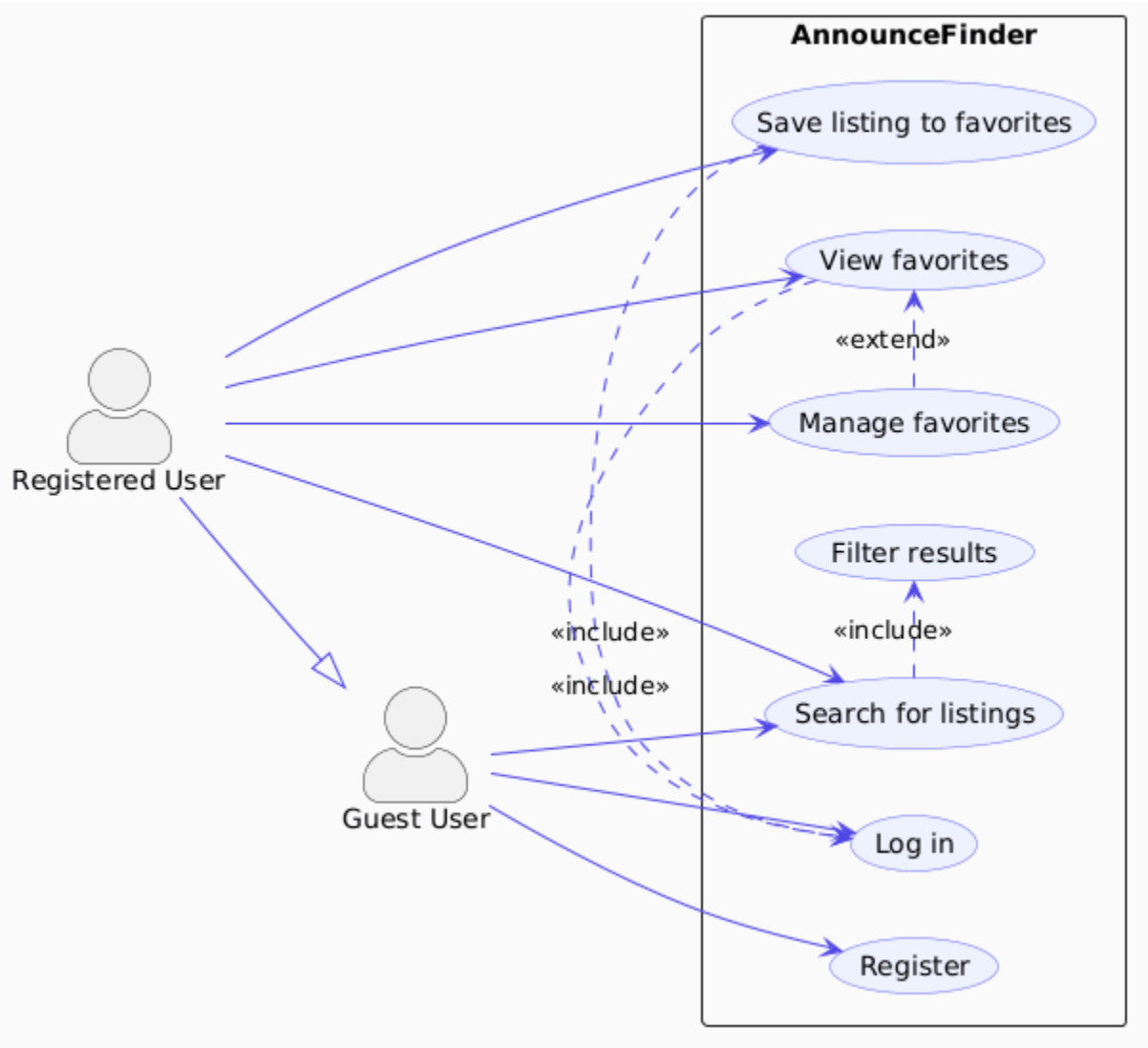

Figure 1: Use Case Diagram

1.5 Supplementary Specification

1.5.1 Non-functional Requirements

- NFR1 - Performance: The system shall return search results within 10 seconds for a standard query across all platforms. This is necessary because scraping involves multiple sequential network requests and users should not wait excessively for aggregated results.
- NFR2 - Reliability: The system shall continue to function and return partial results even if one or more scrapers fail. This is achieved through the plugin architecture where each scraper is isolated and exceptions do not propagate to the rest of the system.
- NFR3 - Security: User passwords shall be stored hashed in the database and never in plain text. Sessions shall be managed securely to prevent unauthorized access to

user favorites.

- NFR4 - Maintainability: The system shall be structured so that new scrapers for additional platforms can be added or existing ones removed without modifying the core application logic. This is achieved through the BaseScraper plugin architecture where each platform is encapsulated in its own class.

1.5.2 Design Constraints

- DC1: The system is built using Python and the Flask web framework, which constrains the backend technology stack.
- DC2: Scraping is limited to publicly accessible pages only. Pages requiring authentication or blocked by robots.txt are excluded.
- DC3: The eBay integration relies on the official eBay Browse API, which requires a registered developer account and is subject to eBay's API usage policies and rate limits.
- DC4: The system runs as a local development server and is not deployed to a production environment in this iteration.
- DC5: Rate limiting is enforced per domain to comply with website terms of service, which limits the speed at which results can be retrieved.
- DC6: Scrapers that rely on HTML parsing are subject to breaking if the target website changes its HTML structure.

1.6 Glossary

- Web Crawler: A software program that automatically browses the web, following links and collecting data from web pages.
- Web Scraper: A program that extracts specific data from the HTML content of a web page.
- Listing: A single product announcement retrieved from a marketplace, containing title, price, platform, image, and URL.
- BaseScraper: The abstract base class that all platform-specific scrapers inherit from, defining the common interface for the search method.
- robots.txt: A file hosted by websites that specifies which pages crawlers are allowed or disallowed from accessing.
- Rate Limiting: The practice of introducing delays between requests to avoid overloading external servers.
- Fuzzy Matching: A technique for comparing strings that allows for partial or approximate matches, used to improve the relevance of search results.
- API (Application Programming Interface): A structured interface that allows software to request data from an external service in a structured format, as opposed to scraping raw HTML.
- Favorites: A user-specific collection of saved listings stored in the database, accessible only after authentication.

2 Deliverable 2

2.1 Domain Model

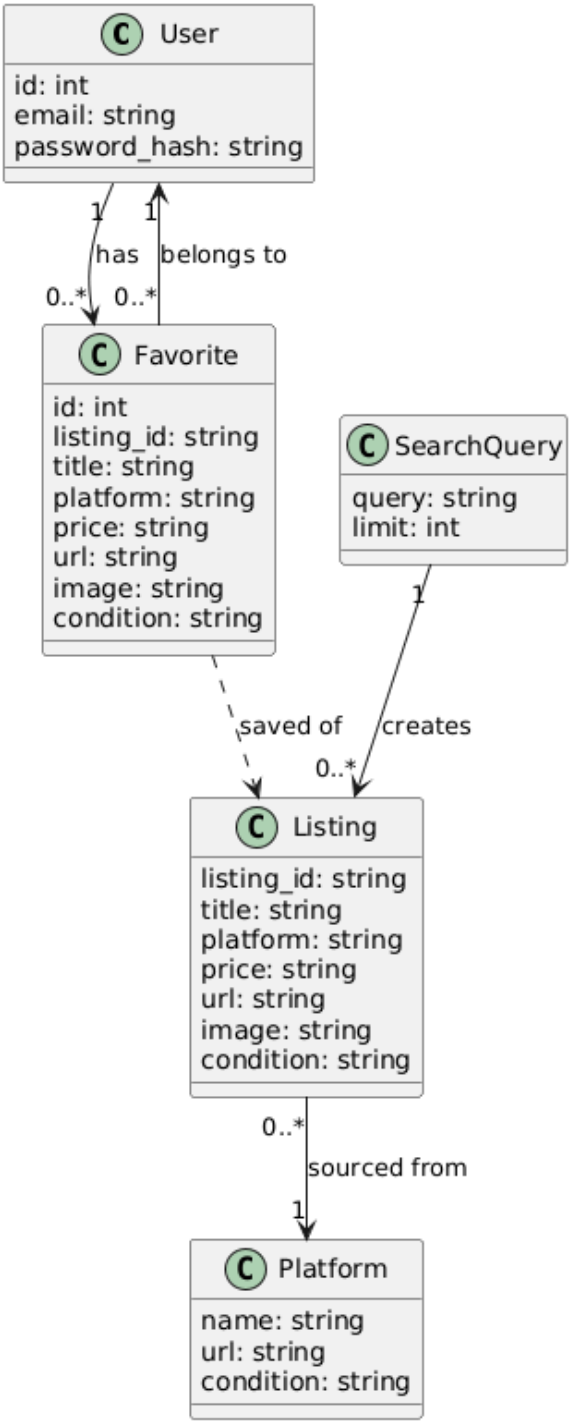

Figure 2: Domain Model Diagram

2.2 Conceptual Architecture

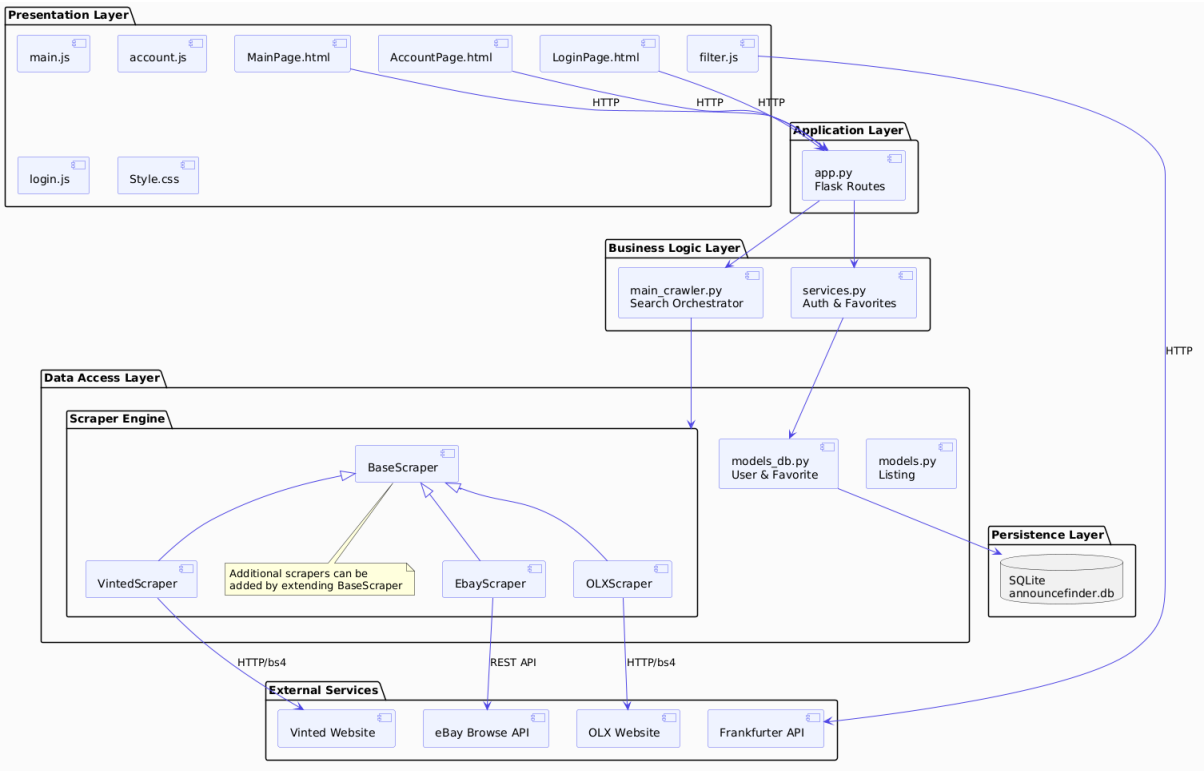

Figure 3: Conceptual Architecture Diagram

2.3 Package Design

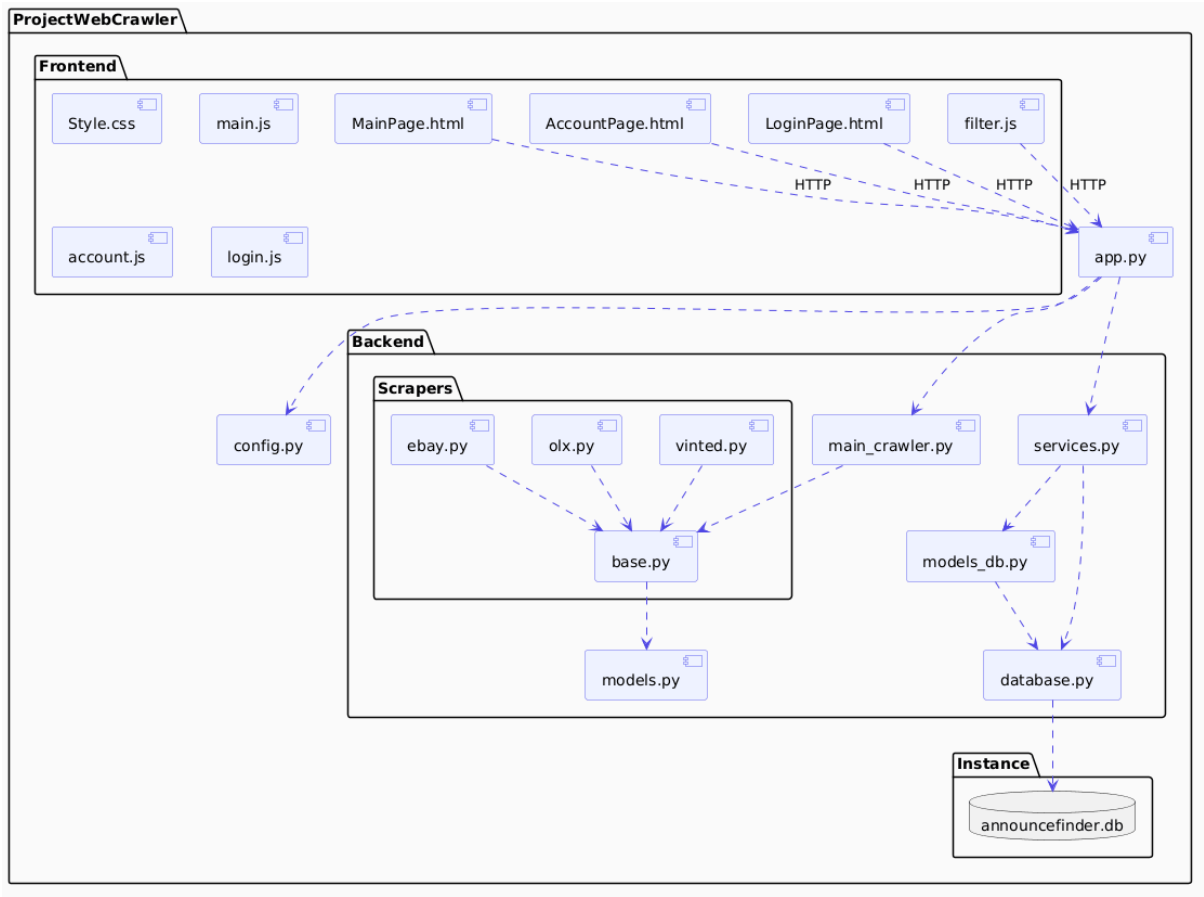

Figure 4: Package Diagram

2.4 Components and Deployment

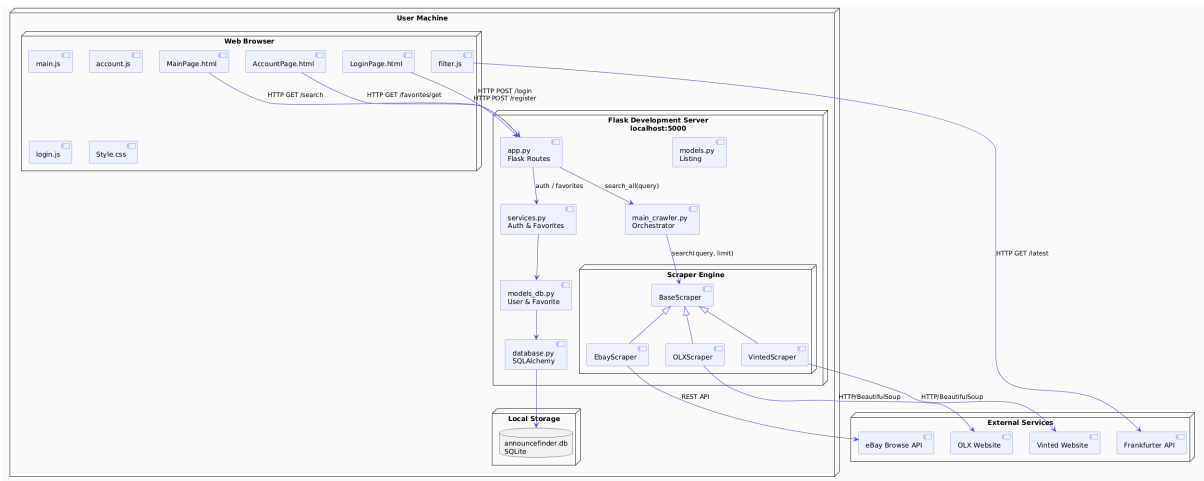

Figure 5: Component and Deployment Diagram

3 Deliverable 3

3.1 Design Model

3.1.1 Dynamic Behavior

[Create the interaction diagrams (2 sequence) for 2 relevant scenarios]

3.1.2 Class Diagram

[Create the UML class diagram; apply GoF patterns and motivate your choice]

3.1.3 Data Model

[Create the data model for the system.]

3.1.4 System Testing

[Describe the testing methods and some test cases.]

3.1.5 Future Improvements

[Present some features that apply to the application scope.]

3.2 Conclusion

3.3 Bibliography